Laboratorio ARSW No.5

María Belén Quintero Aldana

Nikolas Martinez Rivera

1. Revisar el código de configuración de seguridad (`SecurityConfig`) e identificar cómo se definen los endpoints públicos y protegidos.

Dentro del código de SecurityConfig se observan cuatro métodos, iniciando por filterChain.

```
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
```

Este método como su nombre indica es una cadena de filtros, este método recibe un HttpSecurity que actúa como builder para configurar la cadena de filtros, cuando vemos el método authorizeHttpRequests nos damos cuenta que acá se configura el acceso de los endpoints, permitAll hace que para los endpoints especificados no se tenga que mandar un token de autenticación con algo, para hasAnyAuthority el Spring Security pide que el cliente deba mandar un token donde tenga el scope especificado en el hasAnyAuthority, por último en el código tenemos un anyRequest authenticated que básicamente dice que para todo endpoint que no haya sido mencionado, se debe pedir autenticación total sobre la api.

```
http
    .csrf( CsrfConfigurer<HttpSecurity> csrf -> csrf.disable())
    .authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth
        .requestMatchers( ...patterns: "/actuator/health", "/auth/login").permitAll()
        .requestMatchers( ...patterns: "/v3/api-docs/**", "/swagger-ui/**", "/swagger-ui.html").permitAll()
        .requestMatchers( ...patterns: "/api/**").hasAnyAuthority( ...authorities: "SCOPE_blueprints.read", "SCOPE_blueprints.write")
        .anyRequest().authenticated()
    )
    .oauth2ResourceServer( OAuth2ResourceServerConfigurer<HttpSecurity> oauth2 -> oauth2.jwt(Customizer.withDefaults()));
return http.build();
```

2. Explorar el flujo de login y analizar las claims del JWT emitido.

Para el flujo del login se observaron los controladores de la aplicación, allí encontramos que en AuthController allí vemos un endpoint principal definido que es /auth y en sus “subendpoints” está el login, aquel login tiene un verbo HTTP de POST, y lo que el usuario envía a este endpoint de “/auth/login” un login request que es un simple registro con el usuario y la contraseña que ingresa el usuario desde un front por ejemplo, lo que hace el endpoint es utilizar un userservice.

UserService es simplemente un servicio en memoria y tiene dos usuarios creados, en este caso los usuarios “quemados” se pueden ver en la siguiente captura de pantalla.

```
public InMemoryUserService(PasswordEncoder encoder) {  1 usage  Javier Toquica
    this.encoder = encoder;
    this.users = Map.of(
        k1: "student", encoder.encode( rawPassword: "student123"),
        k2: "assistant", encoder.encode( rawPassword: "assistant123")
    );
}

public boolean isValid(String username, String rawPassword) {  1 usage  Javier Toquica
    String hash = users.get(username);
    return hash != null && encoder.matches(rawPassword, hash);
}
```

En base al request enviado ese servicio tiene el método isValid que también se puede ver en la captura, en caso de que en efecto el request sea válido, el endpoint lo que hace es tomar la hora exacta, y a esa hora le suma unos segundos que sería el tiempo que el token duraría desde ese mismo momento. También se procede a instanciar un String que muestra el scope del token, con todo lo anterior el endpoint hace uso de métodos de JwtClaimsSet, precisamente usa builder y pega todas las piezas.

```
Instant now = Instant.now();
long ttl = props.tokenTtlSeconds() != null ? props.tokenTtlSeconds() : 3600;
Instant exp = now.plusSeconds(ttl);

String scope = "blueprints.read blueprints.write";

JwtClaimsSet claims = JwtClaimsSet.builder()
        .issuer(props.issuer())
        .issuedAt(now)
        .expiresAt(exp)
        .subject(req.username())
        .claim( name: "scope", scope)
        .build();
```

Pero el token necesita un encabezado, es por eso por lo que se hace uno y se junta al claims para ahora sí tener un token completo, eso es lo que se manda al return.

```
JwsHeader jws = JwsHeader.with(() -> "RS256").build();
String token = this.encoder.encode(JwtEncoderParameters.from(jws, claims)).getTokenValue();

return ResponseEntity.ok(new TokenResponse(token,  token_type: "Bearer", ttl));
```

Y básicamente así se concluye el flujo del login de un usuario en la implementación del laboratorio.

3. Extender los scopes (`blueprints.read`, `blueprints.write`) para controlar otros endpoints de la API, del laboratorio P1 trabajado.

Aprovechando el código de la primera parte, se pasaron las carpetas, compose y aplication.yml para poder fusionar la seguridad con los endpoints de negocio que había en la parte 1 de Blueprints.

Para que la extensión descopes tenga sentido, hacemos una modificación en AuthController, y es que, en vez de dar el mismo scope a todo usuario que haga login, se traslada la lógica al user service con un nuevo método en el que si se es el Admin le dará unos scopes y si es el student le dará menos scopes. Añadimos un usuario más y nos quedamos con lo siguiente.

```java
public InMemoryUserService(PasswordEncoder encoder) {  1 usage  Javier Toquica *
    this.encoder = encoder;
    this.users = Map.of(
        k1: "student", encoder.encode( rawPassword: "student123"),
        k2: "assistant", encoder.encode( rawPassword: "assistant123"),
            k3: "admin", encoder.encode( rawPassword: "admin123")

    );
}

public String getScopesByUsername(String username) {  1 usage  new *
    return switch (username) {
        case "admin" -> "blueprints.read blueprints.write blueprints.create blueprints.update";
        case "assistant" -> "blueprints.read blueprints.write";
        case "student" -> "blueprints.read";
        default -> "";
    };
}
```

Con esto simplemente en SecurityConfig modificamos los requestmatchers que había en el punto 1 por otros para extender y quedaron como se verá en la siguiente captura.

```java
.authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth
        .requestMatchers( ...patterns: "/actuator/health", "/auth/login").permitAll()
        .requestMatchers( ...patterns: "/v3/api-docs/**", "/swagger-ui/**", "/swagger-ui.html").permitAll()
        .requestMatchers(HttpMethod.GET,  ...patterns: "/api/**").hasAnyAuthority( ...authorities: "SCOPE_blueprints.read", "SCOPE_blueprints.write")
        .requestMatchers(HttpMethod.POST,  ...patterns: "/api/**").hasAuthority("SCOPE_blueprints.create")
        .requestMatchers(HttpMethod.PUT,  ...patterns: "/api/**").hasAuthority("SCOPE_blueprints.update")
        .anyRequest().authenticated()
)
```

Como se puede observar se mantuvieron los endpoints de la documentación sin necesidad de autenticación, por el lado de los demás endpoints, se cambió el enfoque bajo el cual se pide scope, en este caso ya no lo hicimos por la url si no por el verbo HTTP que se quiera aplicar pide ciertos permisos. Cabe aclarar que Los endpoints GET siguen aceptando blueprints.read o blueprints.write, eso significa que tanto student como assistant y admin pueden hacer GET. En cambio, POST exige específicamente blueprints.create y PUT exige blueprints.update, que según los scopes que se definió en el userservice, los tiene el admin.

4. Modificar el tiempo de expiración del token y observar el efecto.

Se procedió a cambiar el tiempo del token en el application.yml

```yaml
blueprints:
  security:
    issuer: "https://decsis-eci/blueprints"
    token-ttl-seconds: 20
```

Code | **Details**

200

Response body


```
{
  "access_token": "eyJhbGciOiJSUzI1NiJ9.eyJpc3MiOiJodHRwczovL2R1Y3Npcy11Y2kvYmx1ZXByaW50cyIsInN1YiI6ImFkbWluIiwiZXhwIjoxNzcxOTkxNDQzLCJpYXQiOjE3NzE5OTE0MjMsInNjb3B1IjoiYmx1ZXByaW50cy5yZWFkIGJsdWVwcmludHMud3JpdGUgYmx1ZXByaW50cy5jcmVhdGUgYmx1ZXByaW50cy51cGRhdGUifQ.UFUB2zPdWC1e6oYBQL-IxuiHoN1fsEDX9fE4bJeKD1wRj2Z31dtYex2K60xZu52wyDWy4-PWGdQbCptyjx0dFcsOak7vnG3tdQECmv99AB9ofk3OENA3A97VwsopXU_YRQRgfMr-8zuorRo-tOE3IFQ3H8assXnPNMJMqkUkjpuKavdxPyOwkJpfs9kvG0r6etvJVKoUOmspMJ_e2zP6MTw5w7XDAkr-KZhhXh7ExJ0bNg9OBnowRzIw0zEVnWeIPkBe10vrkE1fHRgkFXaV1-Ytn2wHBvtsJO6K03NMbHWu6P7rLBKB2W0UmE2jV0PyHobPtPnfX3gnq8pX9Us8QA",
  "token_type": "Bearer",
  "expires_in": 20
}
```


Download

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Wed,25 Feb 2026 03:50:23 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Responses

Como vemos en la respuesta el token en efecto cambio su duración y como efecto en métodos tenemos que cuando no se ejecuta en el tiempo correspondiente obtenemos la respuesta presentada en la captura a continuación (en ella se resalta que el Jwt expiro).

401 *Undocumented*

Error: response status is 401

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-length: 0
date: Wed,25 Feb 2026 04:03:29 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
www-authenticate: Bearer error="invalid_token",error_description="An error occurred while attempting to decode the Jwt: Jwt expired at 2026-02-25T04:02:14Z",error_uri="https://tools.ietf.org/html/rfc6750#section-3.1"
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Mientras que, si colocamos el contenido del Access token en el espacio de Available authorizations que nos brinda swagger obtenemos la siguiente respuesta.

Available authorizations ×

bearer-jwt (http, Bearer)

Authorized

Value: ******

Logout | Close

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/blueprints' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJSUzI1NiJ9.eyJpc3MiOiJodHRwczovL2RlY3Npcy11Y2kvYmx1ZXByaW50cyIsInN1YiI6ImFkbWluIiwiZXhwIjoxNzcxOTkxOTEzLCJpYXQiOjE3NzE5OTE4OTMsInNjb3BlIjoiYmx1ZXByaW50cy5yZWFkIGJsdWVwcml
  -H 'Content-Type: application/json' \
  -d '{
  "additionalProp1": "string",
  "additionalProp2": "string",
  "additionalProp3": "string"
}'
```

Request URL

```
http://localhost:8080/api/blueprints
```

Server response

Code	Details
200	Response body

```
{
  "id": "new",
  "name": "nuevo"
}
```

Download

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Wed,25 Feb 2026 03:58:36 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```